

University of Belgrade
Faculty of Organizational Sciences

Advertisements Tracking Fraud Detection

Final Project from Course *Application of Machine Learning Algorithms*

Student

Aleksa Radojičić 2024/3156

Belgrade, 21st August, 2025.

Table of Contents

- 1. Introduction.....3
- 2. Dataset.....4
- 3. Exploratory Data Analysis.....5
- 4. Base experiments.....8
- 5. Feature Engineering.....9
- 6. Experiments with Derived Columns.....10
- 7. Hyperparameters Tuning.....11
- 8. Conclusion.....15

1. Introduction

In the era of the Fourth Industrial Revolution, marketing has greatly shifted from the traditional approach, which primarily used TV, radio, print ads, billboards as media, the digital marketing can be utilized through vast online social media platforms, such as Instagram, YouTube, on websites (such as banners). Marketing in these times can be more personalized due to high data volume of users generated each day and fed into popular prediction algorithms that based on combination of supervised and unsupervised machine learning models.

TalkingData, China's largest independent big data service platform, covers over 70% of active mobile devices nationwide. They handle 3 billion clicks per day, of which 90% are potentially fraudulent. Their current approach to prevent click fraud for app developers is to measure the journey of a user's click across their portfolio, and flag IP addresses who produce lots of clicks, but never end up installing apps. With this information, they've built an IP blacklist and device blacklist.

This dataset, which is provided from TalkingData on the [Kaggle](#) competition platform in 2018, covers approximately 200 million clicks over 4 days. The goal is to build an algorithm that predicts whether a user will download an app after clicking a mobile app advertisement, which belongs to a binary classification supervised machine learning problem. CRISP-DM methodology was followed, with no deployment phase.

2. Dataset

The hosts provided two important datasets: ``train.csv`` and ``test.csv``. Training set consist of 184,903,890 and testing set of 18,790,469 rows in total. Train set covers rows from to 2017-11-06 14:32:21 to 2017-11-09 16:00:00, while test set rows span from 2017-11-10 04:00:00 to 2017-11-10 15:00:00. This is an important piece of information when splitting the training set to train and evaluation sets, because to properly model the desired output evaluation and test sets must have the same distribution.

Each row of the training data contains a click record, with the following features¹.

- ``ip``: ip address of click.
- ``app``: app id for marketing.
- ``device``: device **type** id of user mobile phone (e.g., iPhone 6 Plus, Samsung Galaxy 10, etc.)
- ``os``: os version id of user mobile phone
- ``channel``: channel id of mobile ad publisher
- ``click_time``: timestamp of click (UTC)
- ``attributed_time``: if user download the app for after clicking an ad, this is the time of the app download
- ``is_attributed``: the target that is to be predicted, indicating the app was downloaded

The test data is similar, with the following differences:

- ``click_id``: reference for making predictions
- ``attributed_time``: not included
- ``is_attributed``: not included

In the training set there were 184,447,044 null values only in the `attributed_time` column, which is only for the rows where `is_attributed` column is False. This is a 'metalabel' column, a datetime column present only for the users who downloaded an app after clicking the advertisement. Of course, this column is not present in the test data.

¹ Categorical features ip, app, device, os, and channel are encoded, which reduces the number of potential columns that could be derived, such as geolocation columns for ip.

3. Exploratory Data Analysis

The number of rows of the training set introduces a quite challenge, because the sheer size of `train.csv` is 7.5GB. Thus, the polars library was used to do data-wise manipulations, because it works quite better for larger datasets than pandas library: it is more memory friendly, which allows loading data with memory greater than available RAM; the computations (queries) evaluate much faster.

I could load all the data and do some simple queries, but model trainings were impossible to perform using scikit-learn's library on my 32GB machine, because of the internal data copying scikit-learn is doing in training and not that great support for polars. Optimizing column data types and converting the plain `train.csv` file to uncompressed parquet didn't solve the problem. I opted to use first 10 million sample.

I took 100,000,000 random sample of the data and took 900,000 records from 8th November as a training set and random samples of 100,000 records from 9th November between 04 PM and 15 PM as a test set. The EDA is done only using the sampled training set [training / train set].

There were only 2,204 positive cases of the target variable `is_attributed`, which is 0.2449%, meaning this is a highly skewed binary classification problem. Thus, accuracy as a classification metric cannot be used as an evaluation metric, but AUC (Area Under the ROC Curve) will be used instead. It's also worth noting AUC is the evaluation metric used for this Kaggle competition.

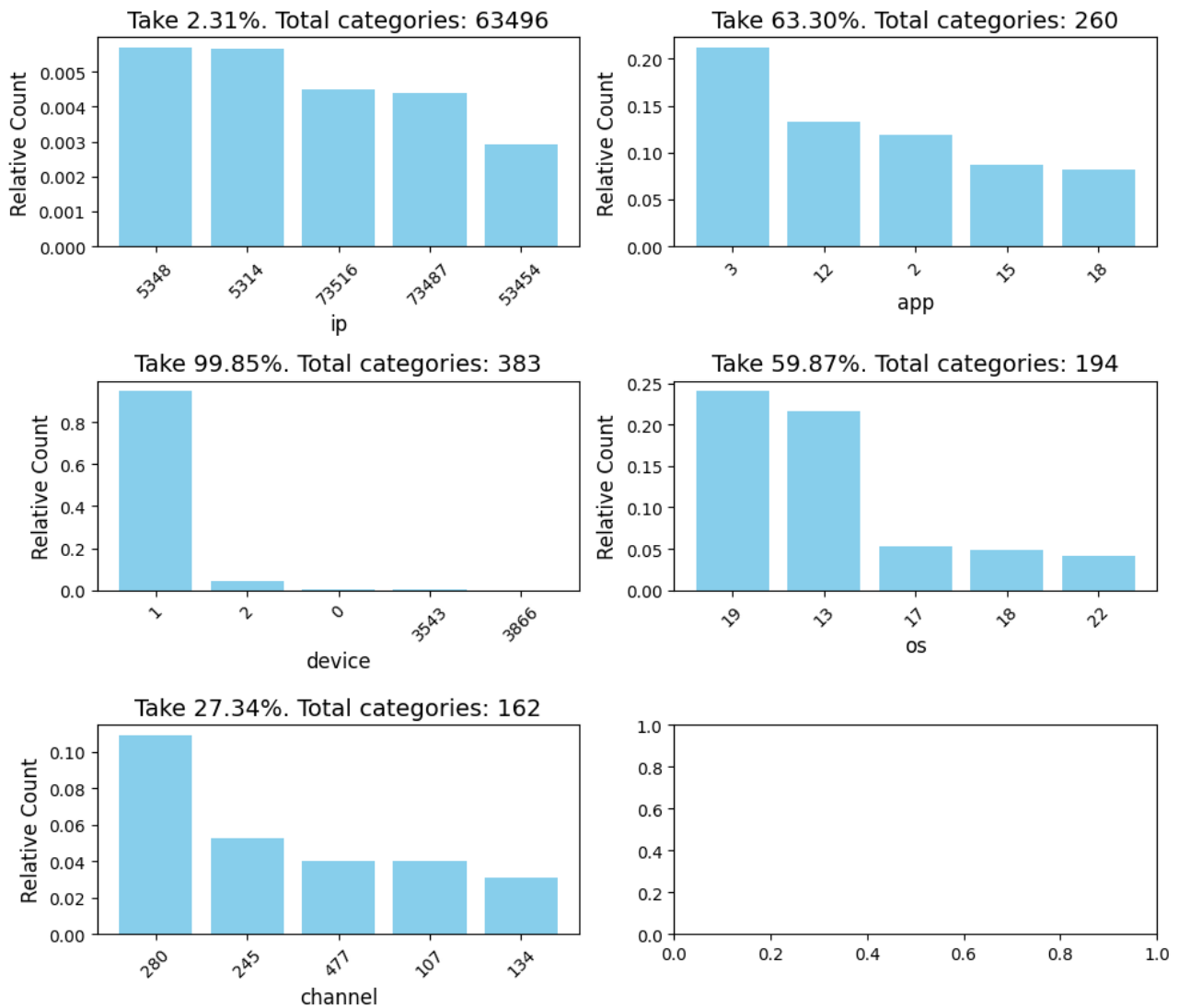
Since the number of rows were quite high, even when sampled, my goal was to start some models as soon as possible, because all the experiments would some time. After the first round of experiments, my idea was to craft more columns, without looking at any Kaggle solution to this competition at all, and do some more EDA along the way with newly inserted columns.

Below is the table showing unique values for each nominal categorical columns:

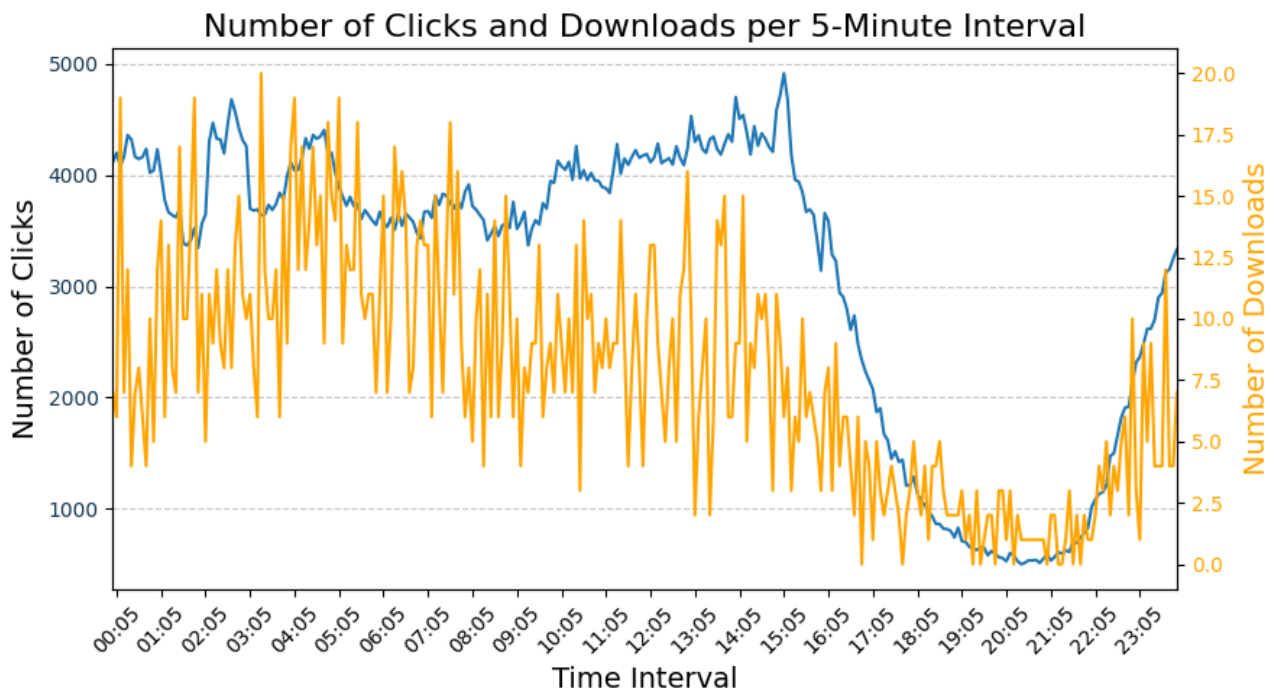
`ip`	63,496
`app`	260
`device`	383
`os`	194
`channel`	162

Column `click\_time` could not be used in its raw, datetime form, as a feature for binary classification, so during feature engineering it was transformed to integer (more in the 5th chapter). Column `attributed\_time` wasn't used in the exploratory analysis in this paper.

Top 5 Categories by Relative Count for Each Feature



For `app` the five apps are dominating with total of 63.30% share. For `ip` instance 5348 had total of 5206 clicks in the 3rd day, suggesting a high traffic. Might be a proxy address or coming from Tor. Must be careful to not include low frequency IPs in training to avoid overfitting. Device type 1 dominates heavily (~95%+). Almost 60% is the share of the top 5 operating systems. Channel column is more diverse, but some channels occur more frequently.



Clicks show a clear daily pattern with a peak in the early afternoon and a sharp decline into the evening. Downloads are very sparse and noisy (only a few per 5-minute bin) compared with the much larger click volumes. Overall clicks and downloads tend to move together, but the low absolute download counts make the relationship noisy.

4. Base Experiments

The first round of experiments involved using columns app, device, os, channel and applying OrdinalEncoder with parameters min_frequency=0.000008, unknown_value=-1, handle_unknown='use_encoded_value', dtype=np.int16. Only tree-based models were used because using OneHotEncoder on models such as LogisticRegression, KNN etc. would do lovely things to my RAM:

Experiment	Classifier	AUC (Train)	AUC (Test)	Hyperparameters
1	Decision Tree	99.774%	74.381%	min_samples_leaf=20
1	Random Forest	99.696%	92.793%	min_samples_leaf=20, n_estimators=100
1	AdaBoost	95.101%	89.665%	min_samples_leaf=20, n_estimators=50
2	Random Forest	99.698%	92.816%	min_samples_leaf=20, n_estimators=200
2	AdaBoost	95.554%	90.513%	min_samples_leaf=20, n_estimators=100
3	Random Forest	99.699%	93.294%	min_samples_leaf=20, n_estimators=300
3	AdaBoost	95.839%	91.444%	min_samples_leaf=20, n_estimators=150
4	Random Forest	99.700%	93.192%	min_samples_leaf=20, n_estimators=400
4	AdaBoost	96.054%	91.982%	min_samples_leaf=20, n_estimators=200

Random Forest with 300 estimators had the highest AUC on the test data from the experiments, while having a much higher AUC on the training set, suggesting a high overfitting.

5. Feature Engineering

The following columns were added in the feature engineering phase:

``click_timestamp`` (uint32): Converting the original click_timestamps to Unix timestamps in milliseconds. Then, normalizing these timestamps by subtracting the first click timestamp and converting the result from milliseconds to seconds.

``previous_sessions`` (uint32): How many previous sessions each IP has had based on the 15 minutes duration between sessions, allowing for analysis of user behavior over time. Also denotes session ID per IP.

``total_sessions`` (uint32): Total sessions user had when the entire set is analyzed per unique (IP, previous_sessions) pair.

``current_session_duration_till_now`` (uint32): Cumulative duration of the current session, calculated based on the time differences between their clicks (click_timestamp). This allows for an analysis of how long users have been active in their current session up to that point per unique (IP, previous_sessions) pair.

``current_session_duration`` (uint32): Current session duration when the entire set is analyzed per unique (IP, previous_sessions) pair.

``avg_previous_sessions_duration`` (float64): Average previous session duration when the entire set is analyzed per unique (IP, previous_sessions) pair. There were 143,232 NaNs (when IP clicks an ad for the first time) and they were filled by -1.

Some insights gained when analyzing the derived columns:

- * Only 25% of IPs had one session.

- * IPs with **11** or more total sessions usually **do not download** according to the 3rd quantile. This analysis is partly complete because it only looks at (``ip``, ``total_session``) pairs and does not include ``app``. Thus, IPs with none downloads and high session could be bots or users that do not care about advertisements products². It could be some users stubbornly avoid downloading a particular particular app, but they are willing to download some other application. 4th quarter IPs downloaded up to 3 times.

- * There are 27.71% IPs with total sessions of 11 or more, however such IPs contribute to **69.39%** of the total application clicks.

² When gaming, watching an ad will give you in-game benefits, so this could explain why someone has a lot of sessions in this short time period, given the fact you could gain perks every 15 minutes in a particular game.

6. Experiments with Derived Columns

Like for base experiments, I used OrdinalEncoder on columns app, device, os, channel with parameters min_frequency=0.000008, unknown_value=-1, handle_unknown='use_encoded_value', dtype=np.int16. Apart from them columns `ip`, `click\_timestamp`, previous_sessions, total_sessions, current_session_duration_till_now, current_session_duration and avg_previous_sessions_duration were propagated without any further preprocessing. The efforts were put on one of the most renowned ensemble models RandomForest and AdaBoost.

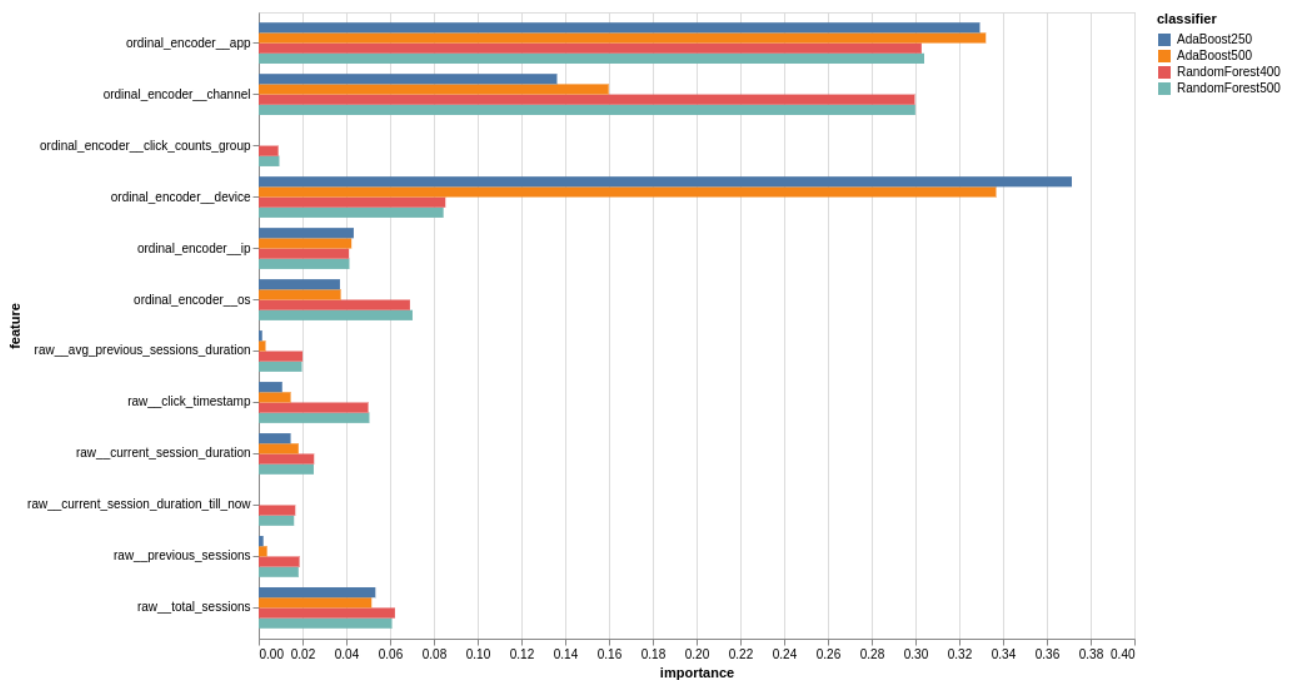
In order for the derived time-based columns to have proper values on the test set, X_train was concatenated temporarily with X_test, so the common IPs in both sets would get correct values. Then, derived columns were made for the concatenated set and X_train records were removed, leaving only X_test.

Here are the results for the two best AdaBoost and RandomForest models:

Experiment	Classifier	AUC (Train)	AUC (Test)	Hyperparameters
3	RandomForest	99.800%	93.006%	min_samples_leaf=20, n_estimators=300
3	AdaBoost	96.274%	92.994%	min_samples_leaf=20, n_estimators=150
4	RandomForest	99.800%	93.167%	min_samples_leaf=20, n_estimators=400
4	AdaBoost	96.491%	93.398%	min_samples_leaf=20, n_estimators=200

Both models overfitted the data, especially RandomForest. AdaBoost managed to get higher AUC score on test data even though the simpler model is used (200 estimators compared to 400 RandomForest estimators).

Below are the feature importances for the 4 models generated from tree models themselves:



Based on the plot RandomForest is able to better use the newly added columns compared to AdaBoost. That is one of the reason RandomForest was used for tuning hyperparameters. IP was insignificant and the difference between usefulness of click_timestamp varies drastically between AdaBoost and RandomForest. Both models agree app and channel were the most responsible for deducing the target label value. From the derived columns `total\_session` and then `click\_timestamp` were proven the most important.

7. Hyperparameters Tuning

OrdinalEncoder is also applied for ordinal features with the same settings as before.

The following grid of hyperparameters is used for tuning the RandomForest algorithm using GridSearch without cross validation³:

Hyperparameter	Values
n_estimators	[300, 400]
max_depth	[6, 10, None]
max_features	['sqrt', 6, 12]
min_samples_leaf	[20]

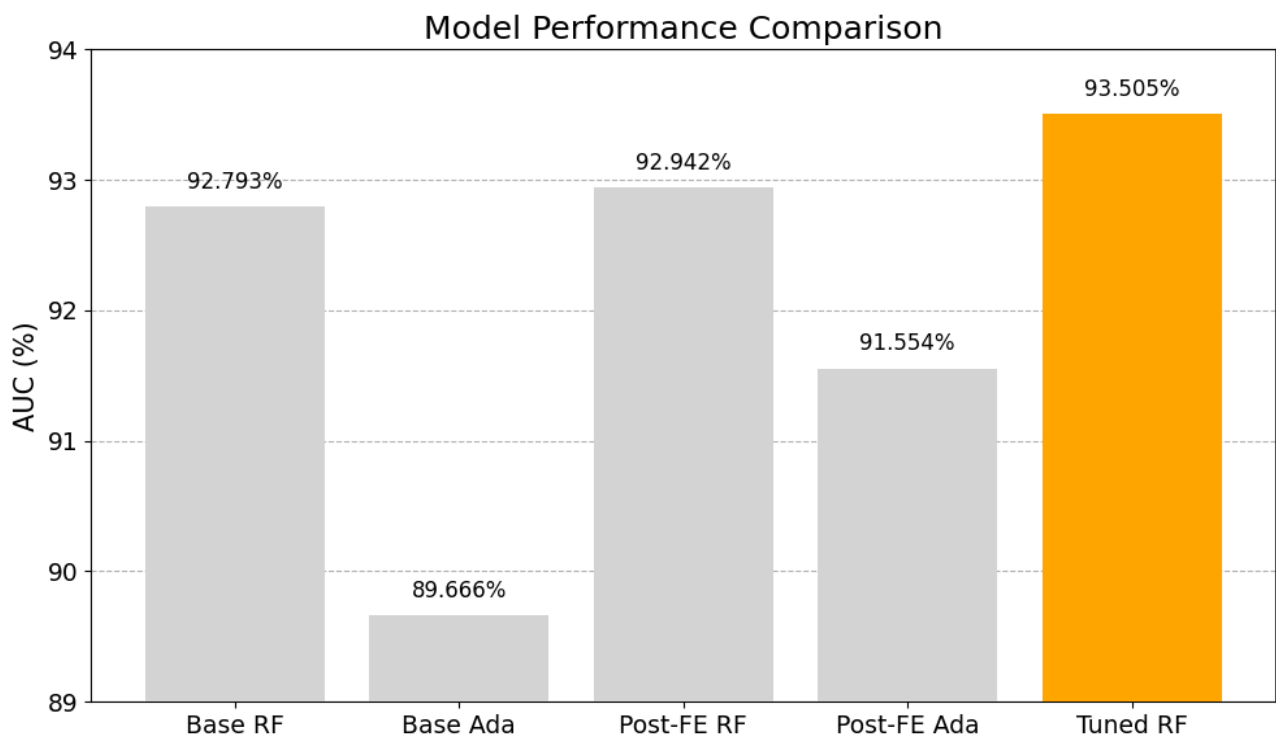
³ Cross validation would be harder to utilize because of the large dataset size and long training.

Here is the optimal selection of the hyperparameters with the results:

max_depth	max_features	min_samples_leaf	n_estimators	train_auc	test_auc
10	12	20	400	98.664%	93.504%

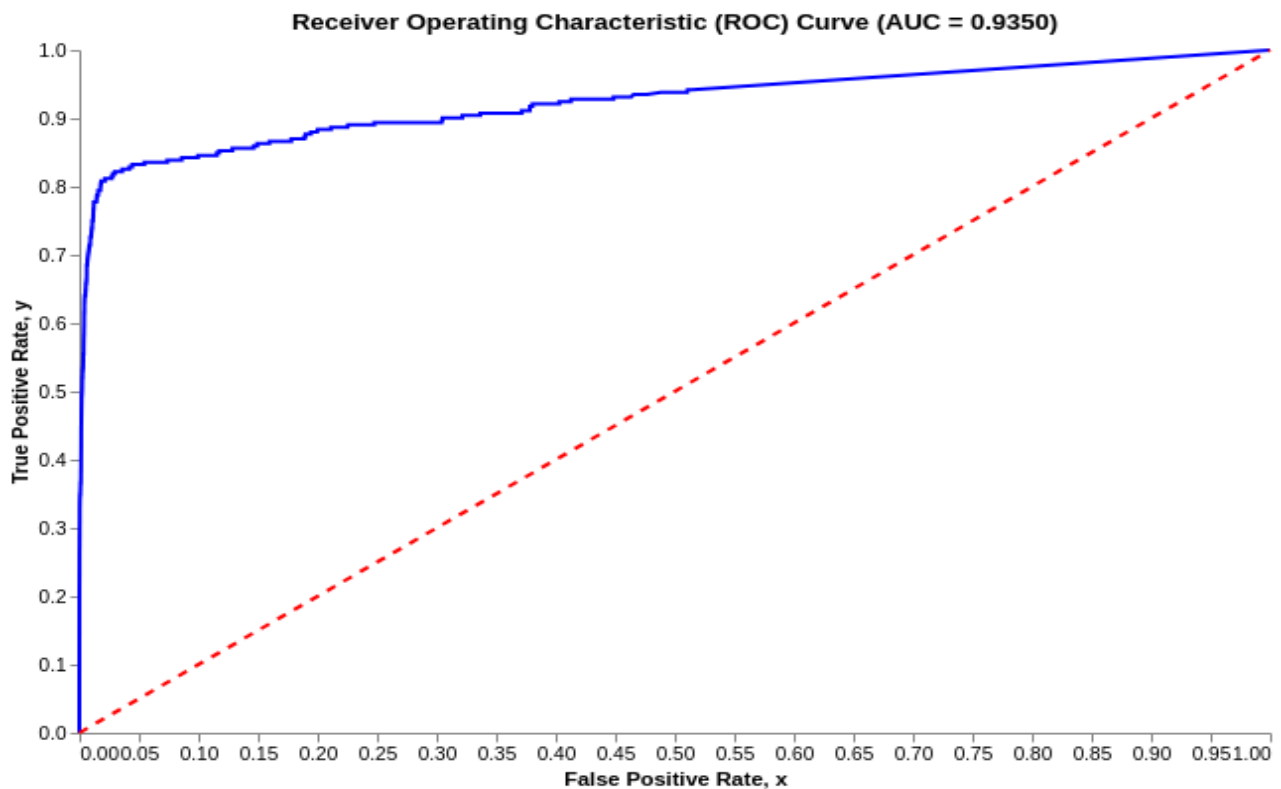
The model also overfitted the training data, but less compared the previous experiment with derived columns.

Below is the comparison plot between base models with original features, base models after feature engineering and the tuned RandomForest model:

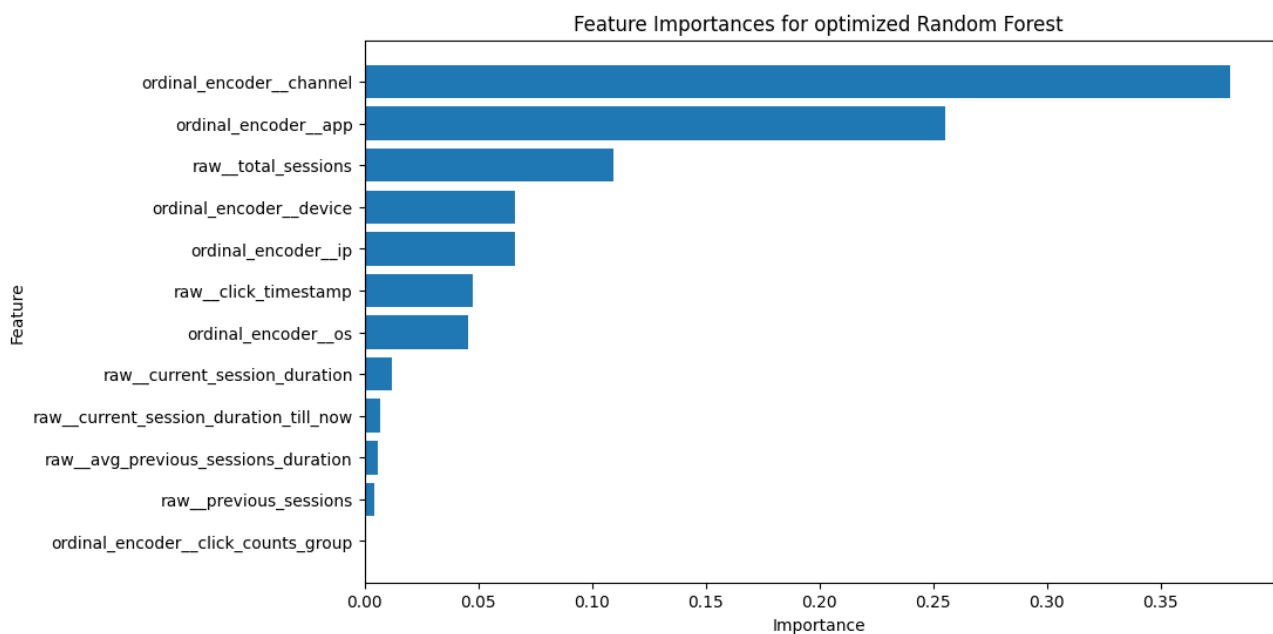


The model is saved locally using joblib package in Python.

Here is the ROC curve for the optimized model:



The feature importances are different compared to the previous experiments. On this plot we can see the derived columns were less impactful than before:



Columns `channel` and `app` continue to be dominantly significant, while third useful was `total\_sessions`.

8. Conclusion

By classifying clicks whether they are successful, i.e. resulted in a download, TalkingData or any other company could classify IP addresses that fault often and prioritize certain channel and apps. Solving the challenge of high number of clicks for particular IP addresses is crucial, because one IP can be shared by multiple users (router IP or proxies). Also, if IP addresses are classified whether they belong to a certain proxy, VPN or Tor Network, it would be easier spotting a bot activity. However, if IP was uniquely identifying user than this would be a more trivial problem.

Tree-based models once again prevailed for detecting application downloads, where RandomForest turned out to be a winner for the used preprocessing, but AdaBoost was also decent, but it more poorly fit the training set. Even the base model achieved the excellent value for AUC, but adding the extra features bumped up the performance.

The best RandomForest model from the performed experiments can be used in production because the result AUC evaluation metric achieved more than 90% on the test set. A robust pipeline with monitoring would be needed to accompany for any distribution changes.

More improvements could be done in the future. Solutions that scored in the top 25 of the Kaggle public leaderboard used primarily LightGBM algorithm, with a lot of derived columns, such as ``previous_click_timestamp`` by (``ip``, ``app``, ``os``, ``device``), ``next_click_timestamp`` by (``ip``, ``app``, ``os``, ``device``) (i.e. click time deltas), click count within next one / six hours, different grouped by features with different column pairs... Column ``attributed_time`` could be also taken into account during EDA, squeezing potentially more performance and increasing the AUC metric.

Since TalkingData in their database contains the real IP addresses, operating systems, channels even more columns can be used in training for a more precise and robust model, such as geolocation columns (``continent``, ``country``, ``district``...), device manufacturer and so on.